

Summer 2026 Research Report

Miraj Parekh

8/13/26

Part 0

Background

Tensegrity structures consist of rigid struts held in place by a network of cables. While tensegrities are naturally resilient from these properties, they are also very difficult to control. With vibrationally actuated tensegrities, small changes in motor frequency can result in big differences in movement, much of it being non-linear and unpredictable.

SPVVVECTR is a 3-motor tensegrity robot that locomotes through vibration. Each motor can have an RPM range between -1000 and 1000 , so testing which inputs have effective locomotive capability is not possible with hand-trials. Also, previous empirical trialing demonstrated that higher RPMs (such as max values) usually led to higher displacements. However, it is my goal to determine the efficacy of a machine learning algorithm such as Bayesian Optimization in generating optimal inputs that empirical testing could not find.

Every input is a 20-second trial throughout our experiments and we decided to make our fitness function the linear displacement over the course of a 20-second trial.

Robots Tested

1. **SPVVVECTR-1a**: First robot, lighter springs and overall lighter weight. Therefore easier to move with lesser RPMs.
2. **SPVVVECTR-1b**: Second robot, heavier springs and overall heavier body. Therefore displacements across the board were less.

Qualisys Motion Capture

All displacement tracking was done utilizing NSF-funded Qualisys motion capture system that recorded infrared signals from markers placed on SPVVVECTR to stream live 6-DOF Pose data.

Then, I programmed Python scripts leveraging the QTM API that makes possible closed-loop optimization trials with SPVVVECTR. Below is a brief description of this loop:

1. The optimizer proposes an RPM-input.
2. The input is sent to SPVVVECTR's controller over BLE.
3. QTM streams live positional data for the duration of the 20-second trial.

4. Net linear displacement from initial position to final position is automatically computed and brought to the attention to the user.
5. The user manually presses **s** to return displacement to optimizer, or **r** to discard and retry trial.
6. If (5) is **s**, the optimizer updates its surrogate model and proposes next input.

Bayesian Optimization

The optimizer was programmed with `scikit-optimize` using the ask-tell `Optimizer` methods. This way, each proposed input could be physically run before the next one was requested.

The surrogate model was a `GaussianProcessRegressor` with a Matérn kernel ($\nu = 2.5$), `normalize_y=True`, and Gaussian noise estimation. Then, the acquisition function was Expected Improvement (EI). The search space was 3 dimensional on the set of Integers between $[-1000, 1000]$ RPM per strut. Since `skopt` minimizes by default, we had to negate our displacement when inputting through `opt.tell()`. LHS Priors were generated with `Lhs(lhs_type="classic")` over the search space. Random priors were uniformly random.

Different Trial Protocols

Conducted over 300 trials, all 20-seconds each. Throughout experimentation, we identified two separate protocol methods to run these 20-second optimization trials. Detailed below are these protocols:

1. **constant-pose**: Here we give SPVVVECTR a constant starting point and orientation that we manually reset following every trial. This protocol became later preferred because variability in displacements was far less of an issue here.
2. **variable-pose**: This was our first protocol that was used for the majority of experiments. Here, we did not interfere as much with resetting the robots position/orientation between trials, unless SPVVVECTR was almost outside camera bounds or too close to an obstacle. Variability in displacement was much more of an issue here because our experiment arena had a slight incline. Therefore two equal RPM-inputs could have vastly different outcomes from marginal differences in starting position and orientation. Despite this, our optimizer delivered optimistic results under these conditions that we'll go over later.

Part 1

Comprehensive Prior Experiment w/ SPVVVECTR-1a

SPVVVECTR-1a was utilized to test the efficacy of different prior-acquisition strategies. My hypothesis was that a more comprehensive prior-batch across the parameter-space meant more context for the optimization steps to therefore find fit inputs more effectively.

Our parameter space can be denoted as follows:

$$P = \{(x, y, z) \in \mathbb{Z}^3 \mid -1000 \leq x, y, z \leq 1000\}$$

My first 3 experiments tested this hypothesis with 3 respective prior-acquisition strategies. Note that these 3 experiments were ran with **variable-pose** protocol:

1. Random Priors
2. Latin Hypercube Sampling
3. Minimal Random Priors (~ 10 randoms)

For (1) and (2), I ran 15 priors and then ran 35 optimization steps.

For (3), the purpose was to avoid priors, and simply run 50 optimization steps. However, because of a few technical issues, the program automatically took 10 random priors to get a baseline. However, this still led to a distinct outcome from the 15 random priors.

Figures 1–3 showcase the different distributions in where the optimizer chose to focus its search. Each axis represents a motor, and the heat (color) represents our fitness, the linear displacement. Tables 1–3 list the top 5 displacing inputs the optimizer produced.

SPVVVECTR-1a | 15 Random Priors + 35 Optimizations

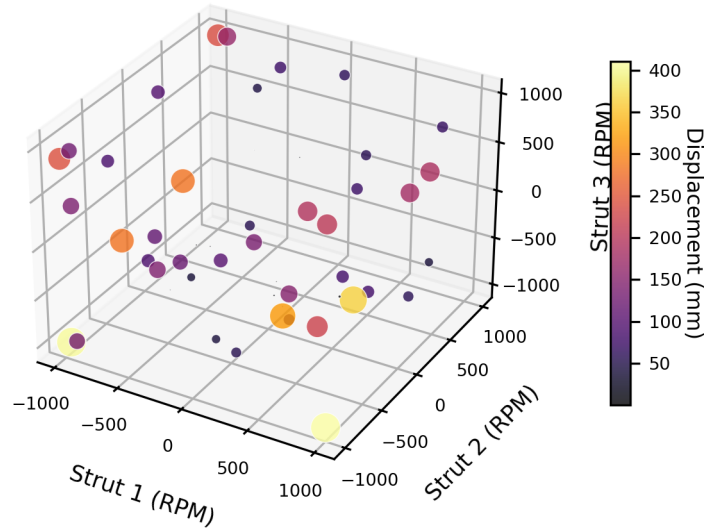


Figure 1: SPVVVECTR-1a | 15 Random Priors + 35 Optimizations

Table 1: Top 5 Best Gaits: SPVVVECTR-1a | 15 Random Priors + 35 Optimizations

Rank	Prior/Opt	Strut 1	Strut 2	Strut 3	Displacement (mm)
1	Optimization	1000	−1000	−1000	410.76
2	Optimization	−974	−970	−991	404.64
3	Optimization	789	−252	−331	364.35
4	Prior	592	−844	−154	318.03
5	Optimization	−847	−508	−196	282.12

SPVVVECTR-1a | 15 LHS Priors + 35 Optimizations

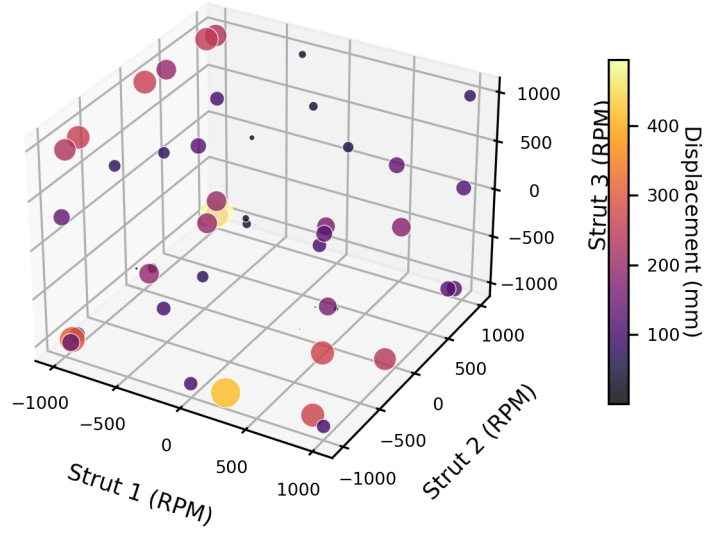


Figure 2: SPVVVECTR-1a | 15 LHS Priors + 35 Optimizations

Table 2: Top 5 Best Gaits: SPVVVECTR-1a | 15 LHS Priors + 35 Optimizations

Rank	Prior/Opt	Strut 1	Strut 2	Strut 3	Displacement (mm)
1	Optimization	-1000	1000	-963	495.06
2	Optimization	258	-989	-989	419.55
3	Optimization	-964	-963	-974	310.05
4	Optimization	979	-987	-269	268.92
5	Optimization	911	-984	-936	268.06

SPVVVECTR-1a | ~ 10 Random Priors + 40 Optimizations

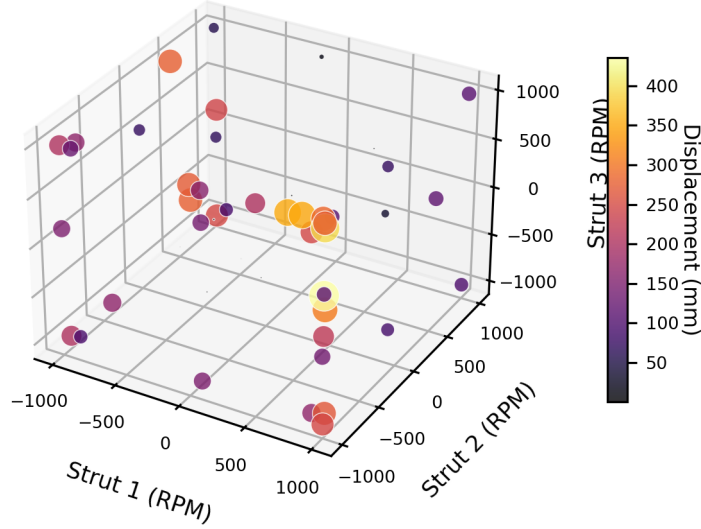


Figure 3: SPVVVECTR-1a | ~ 10 Random Priors + 40 Optimizations

Table 3: Top 5 Best Gaits: SPVVVECTR-1a | ~ 10 Random Priors + 40 Optimizations

Rank	Prior/Opt	Strut 1	Strut 2	Strut 3	Displacement (mm)
1	Optimization	1000	-1000	297	435.57
2	Optimization	993	-984	939	395.49
3	Optimization	734	-1000	1000	352.87
4	Optimization	828	-983	1000	351.37
5	Optimization	997	-980	143	305.52

Takeaway What to takeaway from these experiments are the distribution of inputs our optimizers chose to focus on. Because of further spread in our random and LHS priors, our optimizer searched far more of our parameter-space, P . Although many of these random and LHS priors showcased very weak displacements, our optimizer was able to leverage that information in deciding future observations.

In contrast, our minimal prior experiment showcases a far more pigeon-holed distribution of points because it seems that our optimizer plateaued at a local maximum. Then, most of the search space remains far emptier than the other two strategies.

Figure 4 is a convergence plot that demonstrates this plateau effect for all 3 strategies. This plot illustrates the much quicker ability for the minimal prior strategy to find more optimal displacements in the beginning. However, since the search-space is far less contextualized in this strategy, we run into a concentration of observations in a specific area of the parameter space. Other strategies therefore caught up and generated more optimal displacements a few trials after.

Note that because of the variability of the same input, especially under **variable-pose** protocol, we would have to replicate this experiment with greater sample sizes and **constant-pose** to truly demonstrate the efficacy of LHS priors. However, these experiments led me to primarily utilize LHS priors for future experiments in determining optimal gaits.

Also a side note is how many of the top gaits in each trial hold max RPMs (i.e. 1000, -1000). This follows empirical understanding that higher RPM frequencies cause higher displacements. However, these higher displacements also demonstrate the most variability and unpredictability compared to reduced frequencies. As mentioned in the background, we will get into in part 3 how machine learning may be able to find a ‘sweet-spot’ input that is more predictable than max frequencies, yet also higher displacing than slow frequencies.

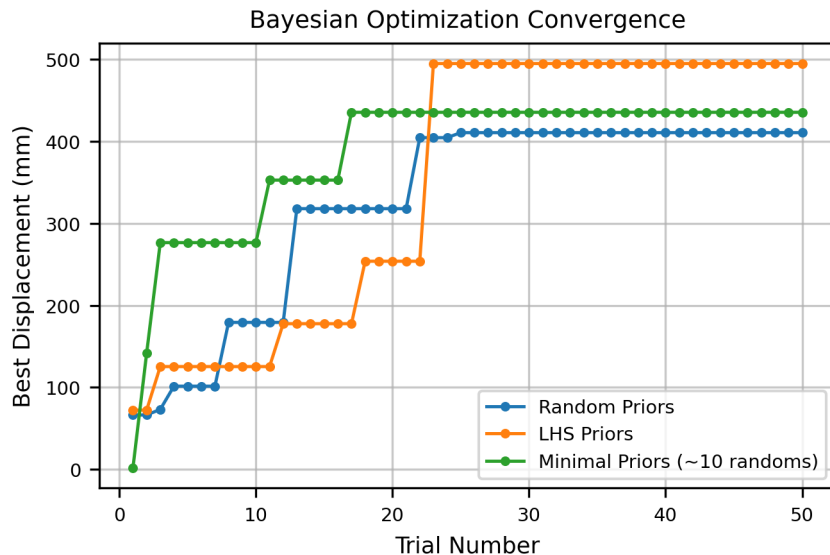


Figure 4: Bayesian Optimization Convergence

Part 2

Protocol Testing w/ SPVVVECTR-1b

Following SPVVVECTR-1a’s experiments, I became more interested in utilizing **constant-pose** for future experiments. I was curious of the variability that the same input caused if it was given the same starting point and orientation for every trial.

For this experiment, we took a baseline vertex input, $[1000, 1000, 1000]$, and ran 2 tests to compare variability between **constant-pose** and **variable-pose** trials. Each had a sample size of 10, and we ran both protocols back-to-back.

Figure 5 is a box plot that illustrates the different outcomes that the same input can lead to depending on the testing protocol. I found that under **constant-pose**, the standard deviation of displacements was cut by more than half than in the **variable-pose** protocol (Table 4).

However, testing with only one constant starting-point misses the goal of a vibrationally-actuated tensegrity like SPVVVECTR, which is to locomote on multiple surfaces and orientations. So while

constant-pose proved more effective for variance testing specific RPM-inputs, I was going to utilize both protocols for my next step in finding optimal gaits for SPVVVECTR-1b.

SPVVVECTR-1b | Protocol Variance at Max Gait [1000, 1000, 1000]

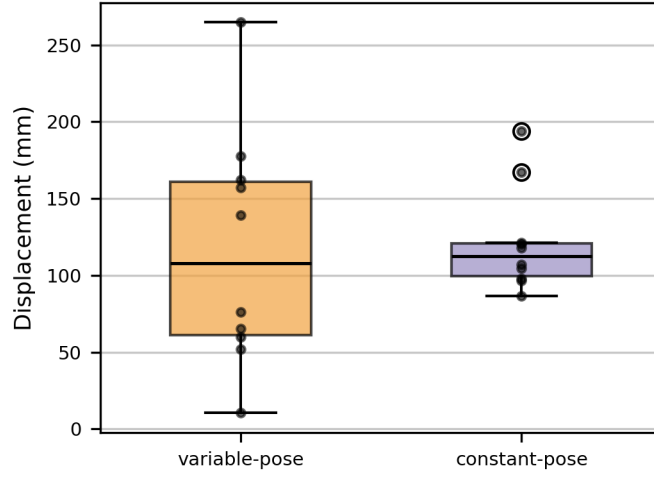


Figure 5: SPVVVECTR-1b | Protocol Variance at Max Gait [1000, 1000, 1000]

Table 4: SPVVVECTR-1b | Protocol Variance at Max Gait [1000, 1000, 1000]

Protocol	n	Mean (mm)	Std (mm)	CV
variable-pose	10	116.42	76.62	0.658
constant-pose	10	121.29	33.79	0.279

Part 3

Generating Inputs & Unexpected Behaviors (SPVVVECTR-1b)

Following our protocol testing, I began running optimizations with LHS priors on our rebuilt robot. I conducted 2 experiments:

1. 15 LHS Priors + 35 Optimization Trials; Sample size of $n = 3$ per trial; **variable-pose**
2. 15 LHS Priors + 35 Optimization Trials; Sample size of $n = 1$ per trial; **constant-pose**

With $n = 3$ for the **variable-pose** experiment, I could account for some of the per-trial outliers that were more likely from varying start positions. In total, 200 20-second trials were conducted for these 2 experiments.

Best Inputs Found

- *constant-pose*) This experiment generated a fittest non-vertex RPM-input at $[886, -24, 479]$ with a respective displacement of 254.78mm.
- *variable-pose*) This experiment’s fittest RPM-input was at $[982, 414, -301]$ with a respective displacement of 253.66mm.

A Closer Look While at first glance it seemed that both fittest inputs were on-par with one another, increased sample sizes for both inputs changed that assumption.

The variance experiments are as follows:

1. $[982, 414, -301]$; $n = 10$; **constant-pose**
2. $[886, -24, 479]$; $n = 20$; **constant-pose**

Figure 6 and Table 5 illustrate results.

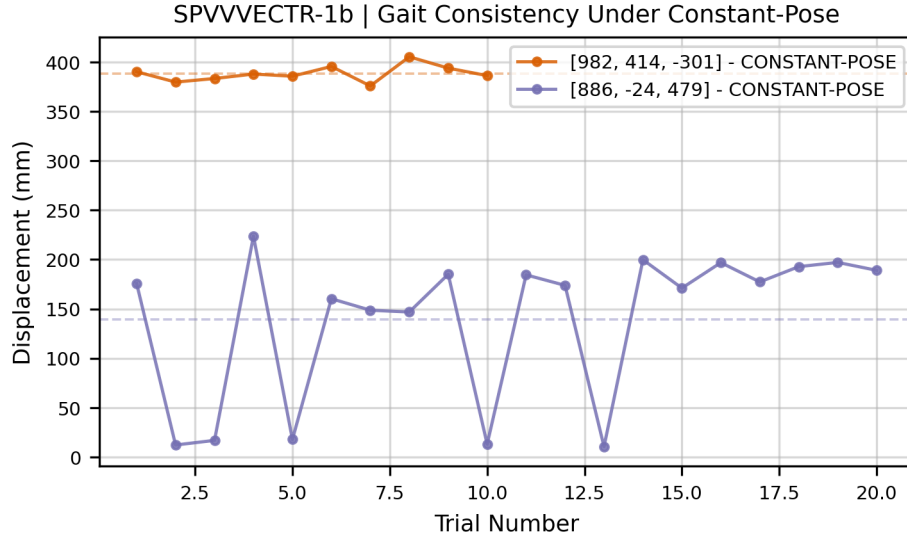


Figure 6: SPVVVECTR-1b | Gait Consistency Under Constant-Pose

Table 5: SPVVVECTR-1b | Gait Consistency Under Constant-Pose

Test	n	Mean (mm)	Std (mm)	CV
$[982, 414, -301]$ – CONSTANT-POSE	10	388.38	8.45	0.022
$[886, -24, 479]$ – CONSTANT-POSE	20	139.69	76.39	0.547

Takeaway and Further Questions Let us denote input $[982, 414, -301]$ as g_a and $[886, -24, 479]$ as g_b .

This data showcased the sheer differences that simply were not present during the Bayesian Optimization trials. I believe part of this is attributed to luck in our choice of orientation of the robot,

since that orientation likely helped g_a leverage its parameters to its fullest potential. It is also important to note that since we used $n = 1$ sample size for the **constant-pose** experiment, that it may have just been an outlier. Overall, we also took away from this experiment that $n = 1$ sampling is not as effective for optimization algorithms.

However, this also raises new questions for how effective g_b could have been if we experimented with multiple orientations, or, if we had built a robot with more motors so that orientation does not matter.

More Importantly g_a proved to be a successful non-vertex input when measuring merely linear displacement, however there was another revolution this input led to.

After a few empirical trials, we found that the behavior of this input was oddly consistent in resembling a linear straight-line motion, despite the fitness function only being linear displacement.

This idea was investigated when incorporating single-axis displacements and yaw-metrics in the tracking scripts. Figure 7 is a concrete visualization of the consistency this input gave.

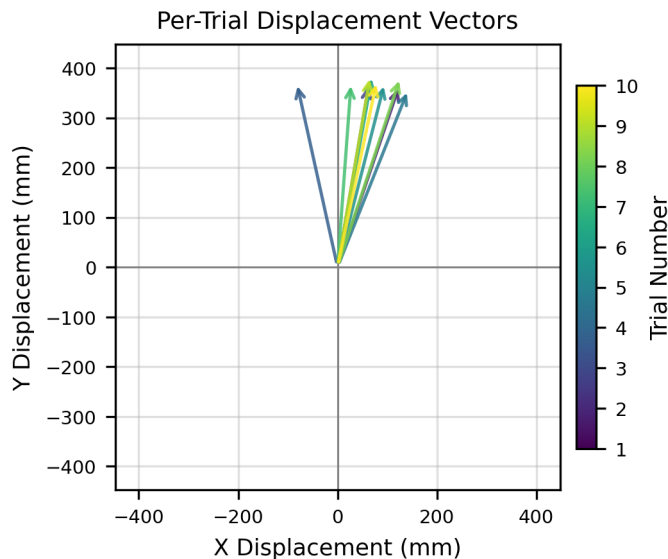


Figure 7: Per-Trial Displacement Vectors

Mean heading: 79.34 deg (std 9.68 deg)

Mean yaw change: -45.10 deg (std 5.59 deg)

This illustrates the efficacy of Bayesian Optimization – that in attempting to maximize linear displacement, the algorithm inadvertently decided that the best way to do so is an input that kept almost a constant 45 degree rotation. On the other hand, inputs empirically found typically fall within 2 categories:

1. Slow RPM; predictable; low displacement.
2. High RPM; unpredictable, constantly changing rotation; usually high displacement.

However with a machine learned RPM-input, we achieved predictability and high displacements.

g_a was a very optimistic finding, and could not have been predicted empirically. Baseline empirical assumptions of optimal inputs usually featured vertex inputs. This also sheds light on eventually achieving single-axis movement-input, and a controlled rotation-input such that we can achieve a controller abstracting away RPM-inputs.

A final experiment afterwards was directly comparing g_a against $[1000, 1000, 1000]$, denoted as g_v .

Part 4

g_a VS g_v

Testing g_a against g_v directly in the same testing session was essential rather than relying on the previous data-set because we find that results can vary depending on the state of battery, and other external variables. To minimize these variables, back-to-back testing was important.

Figure 8 and Table 6 are the results of this comparison, with a sample size $n = 20$ for each.

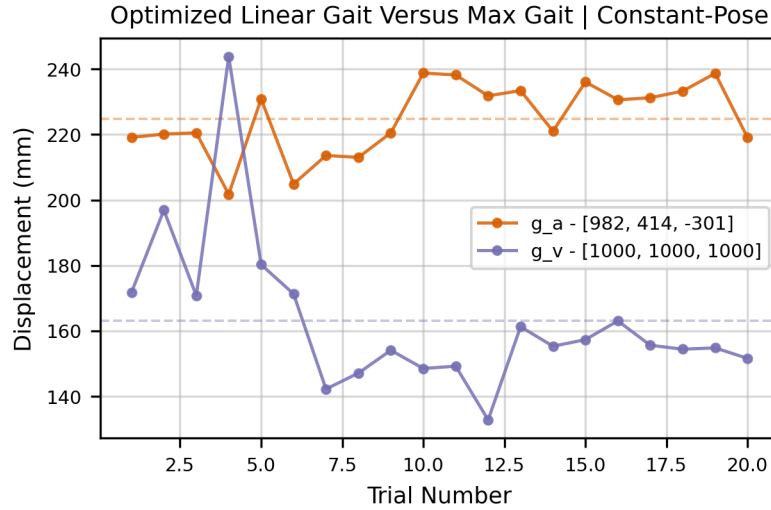


Figure 8: Optimized Linear Gait Versus Max Gait | Constant-Pose

Table 6: Optimized Linear Gait Versus Max Gait | Constant-Pose

Test	n	Mean (mm)	Std (mm)	CV
$g_a - [982, 414, -301]$	20	224.83	11.08	0.049
$g_v - [1000, 1000, 1000]$	20	163.12	23.70	0.145

Takeaway As we see from the data (Table 6), g_a displaces SPVVVECTR-1b 37.8% farther than our baseline g_v and was 2.95x more consistent.

It is worth noting that the g_v outlier value that outperforms every other point in both series demonstrates the unpredictability issue with high frequency RPMs we touched on earlier. Although capable, g_v is far from reliable.

g_a was generated with machine learning, and its ability to beat baseline inputs showcases the efficacy of utilizing Bayesian Optimization in RPM-input prediction.

Part 5

Conclusion & Future Work

With over 300 trials, several experiments, and 2 robots experimented with, the past 8 weeks of research have led to many optimistic results and many more questions.

For future work, I am curious about increasing RPM capabilities to 5000 and also increasing the number of motors. At the moment, orientation matters heavily, so creating a uniform motor setup may alleviate some of the placement issues we faced. Beyond this, I believe other machine learning algorithms can be leveraged as well with SPVVVECTR. Previous research at the UnionEvoLab worked with MAP-Elites to create a map of input behaviors. However, one of the newly found goals I am to achieve is creating a controller and abstracting away from manual RPM-inputs. I believe that altering our optimizer's fitness function from linear displacement has the capability to output desired inputs that we can leverage and make a map from. Simply, all we would need is a rotation gait and single-axis gait.

Overall, there is much potential in the field of vibrational tensegrities, and 8 weeks of research has been an invaluable experience for my discovery of new fields, my ability to problem-solve, my collaboration skills, and my personal curiosity.